LOCAL, GLOBAL, AND STATIC LOCAL VARIABLES

ALVINA ANJUM



SCOPE OF A VARIABLE

- *A variable can be declared as a local, a global, or a static local in C++.*
- The *scope of a variable* is the part of the program where the variable can be referenced.
- **Variable defined inside a function is referred to as a *local variable*.**
- C++ also allows you to use *global variables*. They are declared outside all functions and are accessible to all functions in their scope.
- Local variables do not have default values, but global variables are defaulted to zero.

SCOPE OF A VARIABLE

- A variable must be declared before it can be used.

- The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable.

- The scope of a global variable starts from its declaration and continues to the end of the program.

- A parameter is actually a local variable. The scope of a function parameter covers the entire function.

EXAMPLE DEMONSTRATES THE SCOPE OF LOCAL AND GLOBAL VARIABLES.

function prototype

global variable

local variable

increment x
increment y

local variable

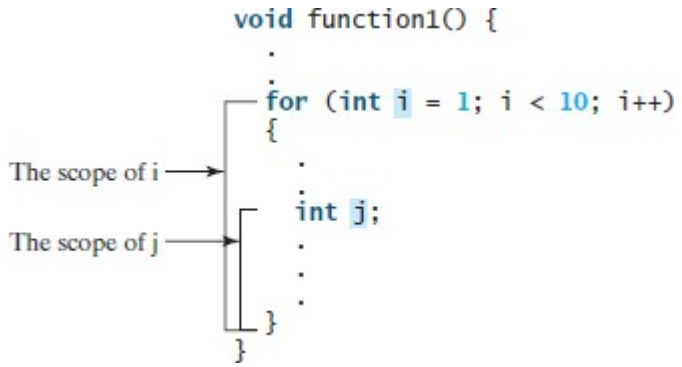
```
1  #include <iostream>
2  using namespace std;
3
4  void t1(); // Function prototype
5  void t2(); // Function prototype
6
7  int main()
8  {
9    t1();
10   t2();
11
12   return 0;
13 }
14
15 int y; // Global variable, default to 0
16
17 void t1()
18 {
19   int x = 1;
20   cout << "x is " << x << endl;
21   cout << "y is " << y << endl;
22   x++;
23   y++;
24 }
25
26 void t2()
27 {
28   int x = 1;
29   cout << "x is " << x << endl;
30   cout << "y is " << y << endl;
31 }
```

output

```
x is 1
y is 0
x is 1
y is 1
```

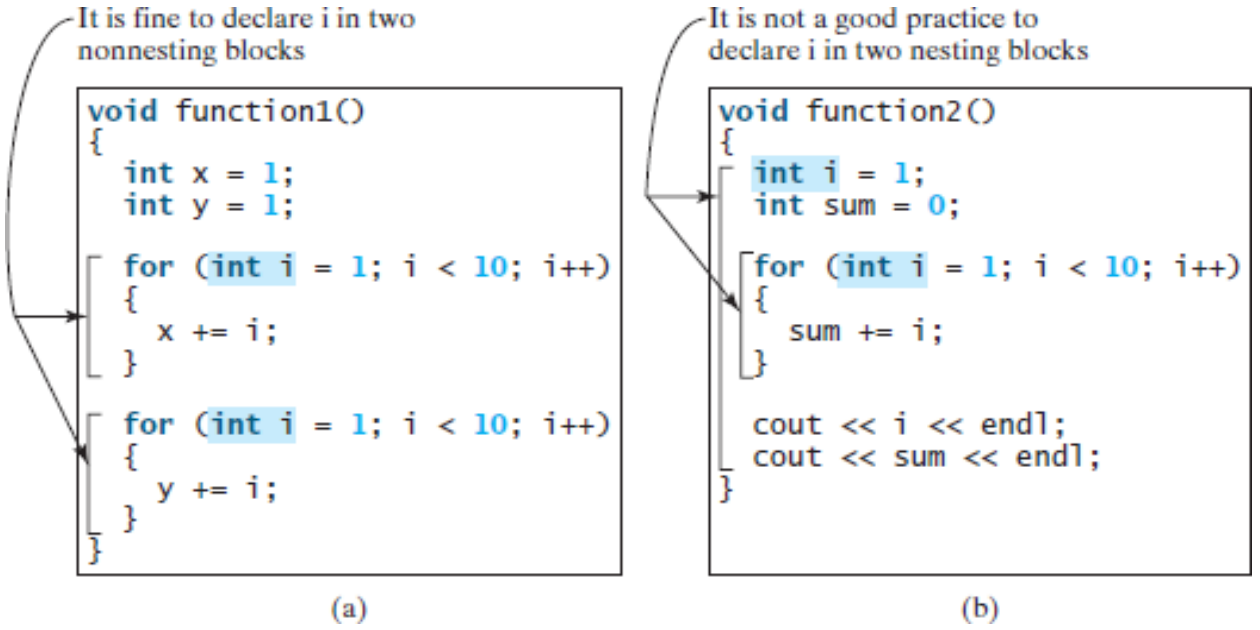
THE SCOPE OF VARIABLES IN A FOR LOOP

- A variable declared in the initial-action part of a for-loop header has its scope in the entire loop.
- However, a variable declared inside a for-loop body has its scope limited in the loop body from its declaration to the end of the block that contains the variable, as shown in



THE SCOPE OF VARIABLES IN A FOR LOOP

- A variable can be declared multiple times in nonnesting blocks, but you should avoid declaring them in nesting blocks.



THE SCOPE OF VARIABLES IN A FOR LOOP

Caution

- Do not declare a variable inside a block and then attempt to use it outside the block.

Here is an example of a common mistake:

```
for (int i = 0; i < 10; i++)
{
}
cout << i << endl;
```

- The last statement would cause a syntax error, because variable i is not defined outside the for loop.

STATIC LOCAL VARIABLES

- After a function completes its execution, all its local variables are destroyed. These variables are also known as *automatic variables*. Sometimes it is desirable to retain the values stored in local variables so that they can be used in the next call.
- C++ allows you to declare static local variables. *Static local variables* are permanently allocated in the memory for the lifetime of the program. To declare a static variable, use the keyword static.

EXAMPLE DEMONSTRATES USING STATIC LOCAL VARIABLES.

function prototype

invoke t1

static local variable
local variable
increment x
increment y

```
1  #include <iostream>
2  using namespace std;
3
4  void t1(); // Function prototype
5
6  int main()
7  {
8    t1();
9    t1();
10
11   return 0;
12 }
13
14 void t1()
15 {
16   static int x = 1;
17   int y = 1;
18   x++;
19   y++;
20   cout << "x is " << x << endl;
21   cout << "y is " << y << endl;
22 }
```

output

```
x is 2
y is 2
x is 3
y is 2
```

CLASS ACTIVITY: PRINT OUTPUT

```
#include<iostream>
using namespace std;
void inc(int a)
{
    a++;
    cout<<"value of a in inc function = "<<a<<endl;
}
main()
{
int a = 5;
inc(a);
cout<<"value of a in main function"<<a;

}
```

```
value of a in inc function = 6
value of a in main function5
--------------------------------
```

NEXT LECTURE

- Default Arguments
- Inline Functions
- Function Overloading
- Call by value, Call by Pointers, and Call by reference